MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 3

Topic: Empirical analysis of algorithms: Depth First Search (DFS), Breadth First Search (BFS)

Elaborated by:
st. gr. FAF-241
Aurelian-Mihai Tihon

Verified by:
univ. assist.
Fiștic Cristofor

Chișinău – 2026

Contents

1 INTRODUCTION **2**

1.1 Objective 2

1.2 Properties of input data 2

1.3 Chosen metrics 2

2 EMPIRICAL ANALYSIS OF ALGORITHMS **3**

2.1 Algorithm 1: Breadth First Search (BFS) 3

2.2 Algorithm 2: Depth First Search (DFS) 3

2.3 Algorithm 3: A* Search (Optimal Heuristic) 5

2.4 Algorithm 4: Greedy BFS (Speed Heuristic) 5

3 THEORETICAL COMPLEXITY COMPARISONS **7**

3.1 Best-case and worst-case scenarios 7

3.2 Complexity table 7

4 EMPIRICAL BENCHMARK RESULTS **8**

5 CONCLUSION **12**

1. INTRODUCTION

1.1. Objective

The objective of this laboratory work is to study, implement, and empirically analyze graph traversal algorithms. The primary focus is on Depth First Search (DFS) and Breadth First Search (BFS), including both basic and optimized variants. Additionally, two heuristic-based searching algorithms, A* Search and Greedy BFS, were implemented to expand the comparative analysis.

An interactive graph traversal laboratory was developed using Python and Pygame, featuring real-time graphical representation of the search frontier, dynamic wall placement, audio feedback mapped to target proximity, and memory tracking. The application allows for deep empirical analysis across four distinct obstacle distributions. The full source code for this laboratory can be found on GitHub at: `https://github.com/aurelian-th/AA/commit/4931bd083de5c18d9a7233620dea058a4f714156`.

1.2. Properties of input data

For graph traversal and pathfinding algorithms, the "input data" is not a simple array, but rather the structural topology of the environment. In this laboratory, the graph is represented as an unweighted 2D grid where each open cell acts as a node connected to up to four neighbors (N, S, E, W).

Several key properties of this input data drastically alter algorithmic performance: In an entirely empty grid, the effective branching factor is nearly 4. Algorithms like BFS will grow their frontier exponentially ($O(b^d)$). In dense environments or strict mazes, the branching factor is restricted to 1 or 2, severely limiting frontier growth. The spatial distribution of walls dictates how an algorithm performs. I analyzed four variants: empty (0% density), sparse (25% density), dense (45% density), and strict maze walls. Grids are heavily cyclical graphs. Unlike pure trees, a grid allows multiple paths to the same node. A robust `visited` set is mandatory to prevent infinite loops, directly impacting the space complexity.

1.3. Chosen metrics

The algorithms were evaluated based on **execution time** (total wall-clock seconds to find the target), **nodes visited** (efficiency of the search expansion), and **maximum memory** (the maximum number of nodes simultaneously held in the frontier queue or recursion stack). Space complexity is particularly important in search algorithms, as maintaining the frontier is often the primary bottleneck on large maps.

2. EMPIRICAL ANALYSIS OF ALGORITHMS

2.1. Algorithm 1: Breadth First Search (BFS)

Breadth First Search explores the graph in "layers" radiating outwards from the start node. It guarantees the absolute shortest path in an unweighted graph because it evaluates all nodes at depth d before moving to depth $d+1$.

The Basic BFS uses a standard Python `list` as its queue. However, extracting from the front of a list (`queue.pop(0)`) requires shifting all subsequent elements, resulting in an $O(N)$ operation per step. The Optimized BFS uses `collections.deque`, which is implemented as a doubly linked list, reducing the pop operation to a constant $O(1)$ time. Memory scales quickly in wide-open spaces as the perimeter of the search area expands uniformly.

```python
def bfs_optimized(self):
    queue = deque([self.start_node])
    self.visited_set.add(self.start_node)

    while queue:
        self.max_memory = max(self.max_memory, len(queue))
        current = queue.popleft()
        self.frontier_set.discard(current)

        yield from self.update_display(current)
        if current == self.target_node:
            self.found = True
            return

        for neighbor in self.get_neighbors(current[0], current[1]):
            if neighbor not in self.visited_set:
                self.visited_set.add(neighbor)
                self.frontier_set.add(neighbor)
                queue.append(neighbor)
```

Listing 1: BFS (optimized with deque)

2.2. Algorithm 2: Depth First Search (DFS)

Depth First Search dives as deep as possible along a single path before backtracking. It makes no guarantees about finding the shortest path; it simply finds the *first* path.

The Basic DFS relies on the system call stack (recursion). While elegant, Python natively limits recursion depth (default 1000). On a 300×300 grid, paths can easily exceed 90,000 steps, resulting in catastrophic `RecursionError` failures. The Optimized DFS uses an explicit stack (a `list` operating via `append` and `pop`), preventing stack overflows and allowing deep empirical memory tracking. DFS is generally memory-efficient in sparse graphs because it only stores the current path, but execution time varies wildly based on whether it guesses the correct initial direction.

```python
def dfs_optimized(self):
    stack = [self.start_node]

    while stack:
        self.max_memory = max(self.max_memory, len(stack))
        current = stack.pop()

        if current not in self.visited_set:
            self.visited_set.add(current)
            yield from self.update_display(current)

            if current == self.target_node:
                self.found = True
                return

            for neighbor in reversed(self.get_neighbors(current[0], current[1])):
                if neighbor not in self.visited_set:
                    stack.append(neighbor)
```

Listing 2: DFS (optimized iterative)

2.3. Algorithm 3: A* Search (Optimal Heuristic)

A* Search uses a Priority Queue to combine the benefits of BFS (optimality) and Greedy approaches (speed). It calculates the cost to reach a node $g(n)$ plus a heuristic estimate to the target $h(n)$ (Manhattan distance in our grid). By always expanding the node with the lowest $f(n) = g(n) + h(n)$, A* perfectly zeros in on the target while avoiding walls, guaranteeing the shortest path with significantly fewer visited nodes than BFS.

```
def astar_search(self):
    count = 0
    pq = [(0, count, self.start_node)]
    self.visited_set.add(self.start_node)
    g_score = {self.start_node: 0}

    while pq:
        self.max_memory = max(self.max_memory, len(pq))
        current_f, _, current = heapq.heappop(pq)

        # ... logic omitted for brevity ...

        for neighbor in self.get_neighbors(current[0], current[1]):
            tentative_g_score = g_score[current] + 1
            if neighbor not in g_score or tentative_g_score < g_score[neighbor]:
                g_score[neighbor] = tentative_g_score
                h_score = abs(neighbor[0] - self.target_node[0]) + abs(neighbor
    [1] - self.target_node[1])
                f_score = tentative_g_score + h_score

                if neighbor not in self.visited_set:
                    self.visited_set.add(neighbor)
                    count += 1
                    heapq.heappush(pq, (f_score, count, neighbor))
```

Listing 3: A* Search (core logic)

2.4. Algorithm 4: Greedy BFS (Speed Heuristic)

Greedy BFS acts purely on instinct. It uses a Priority Queue ordered solely by $h(n)$ - the estimated distance to the target. It ignores the path cost $g(n)$ entirely. In open environments, it shoots straight to the target incredibly fast. However, if it hits a wall, it is forced to scan the perimeter of the obstacle, often resulting in highly sub-optimal "hugging" paths.

```
def greedy_search(self):
    count = 0
    pq = [(0, count, self.start_node)]
    self.visited_set.add(self.start_node)

    while pq:
        self.max_memory = max(self.max_memory, len(pq))
```

```
        _, _, current = heapq.heappop(pq)

        # ... display logic omitted ...

        if current == self.target_node:
            self.found = True
            return

        for neighbor in self.get_neighbors(current[0], current[1]):
            if neighbor not in self.visited_set:
                self.visited_set.add(neighbor)
                h_score = abs(neighbor[0] - self.target_node[0]) + abs(neighbor
    [1] - self.target_node[1])
                count += 1
                heapq.heappush(pq, (h_score, count, neighbor))
```

Listing 4: Greedy BFS (core logic)

3. THEORETICAL COMPLEXITY COMPARISONS

3.1. Best-case and worst-case scenarios

While the asymptotic notation provides a mathematical bound, the practical performance of these algorithms relies entirely on the target's placement and the environmental layout.

- **Breadth first search (BFS):** The best case occurs when the target is immediately adjacent to the start node, allowing BFS to find it in $O(1)$ time. The worst case occurs when the target is at the maximum possible depth (ex: the exact opposite corner of a massive empty grid). Because BFS expands uniformly like a ripple, it will be forced to traverse nearly every other node in the graph and store a massive perimeter of nodes in memory before reaching the end.

- **Depth first search (DFS):** DFS's best case is when the target lies directly along the first path the algorithm chooses to explore, yielding a direct, instantly found route. The worst case occurs in an empty grid when the target is adjacent to the start node, but DFS selects the opposite direction first. It will blindly dive line-by-line through the entirety of the empty grid, exploring every single available node before finally backtracking to the start to check the adjacent target.

- **Heuristic searches (A* and Greedy):** The best case is an entirely empty grid where the heuristic (Manhattan distance) guides the algorithm in a perfect, unobstructed straight line to the target. The worst case involves "concave" obstacles—such as a large U-shaped wall facing the start node. The heuristic pulls the search directly into the center of the trap. Greedy BFS will waste immense time hugging the inner walls, while A* will be forced to exhaustively evaluate every node inside the "cup" until its $g(n)$ path cost grows high enough to convince it to backtrack and route around the outside of the obstacle.

3.2. Complexity table

The table below summarizes the theoretical Time and Space complexities, where V represents Vertices (open cells) and E represents Edges (connections). It is important to highlight **space complexity**. Graph traversal heavily taxes memory. While most sorting algorithms operate in $O(1)$ space, BFS and A* require $O(V)$ auxiliary memory in the worst case to store the frontier queue.

Table 1: Complexity Analysis of Searching Algorithms

Algorithm	Time Complexity	Space Complexity	Optimal Path?	Primary Data Structure
BFS (Basic)	$O(V+E)^*$	$O(V)$	Yes	List Queue
BFS (Optimized)	$O(V+E)$	$O(V)$	Yes	Deque
DFS (Basic)	$O(V+E)$	$O(V)$	No	Call Stack
DFS (Optimized)	$O(V+E)$	$O(V)$	No	Explicit Stack
A* Search	$O(E \log V)$	$O(V)$	Yes	Priority Queue / Heap
Greedy BFS	$O(E \log V)$	$O(V)$	No	Priority Queue / Heap

*Basic BFS suffers heavy constants due to $O(N)$ queue shifting.

4. EMPIRICAL BENCHMARK RESULTS

Using the developed Python application, the algorithms were benchmarked across sizes from 50×50 to 300×300. Recursive DFS predictably crashed on larger grids, represented by blank spaces or extreme spikes.

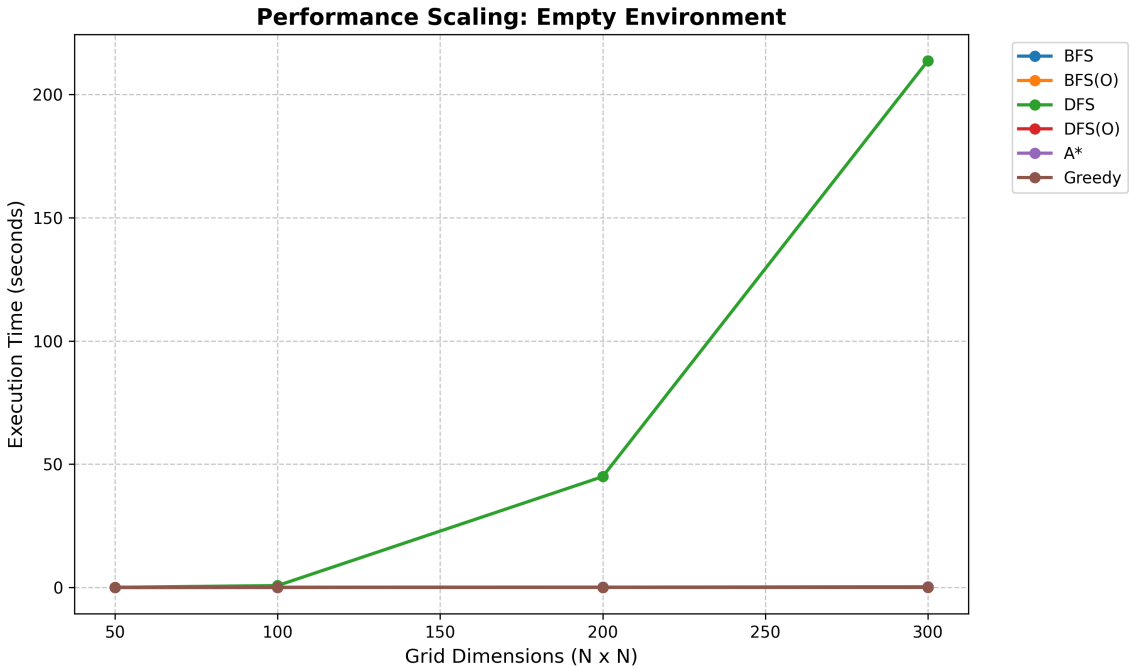

Figure 1: Algorithm performance scaling on an empty grid

[Distribution: EMPTY]

Size	BFS	BFS(O)	DFS	DFS(O)	A*	Greedy
50x50	0.0036	0.0049	0.0466	0.0038	0.0074	0.0004
100x100	0.0199	0.0197	0.7627	0.0160	0.0278	0.0007
200x200	0.0865	0.0794	44.9560	0.0689	0.1276	0.0011
300x300	0.1889	0.1608	213.6241	0.2172	0.3137	0.0017

Figure 2: Raw execution times: empty environment

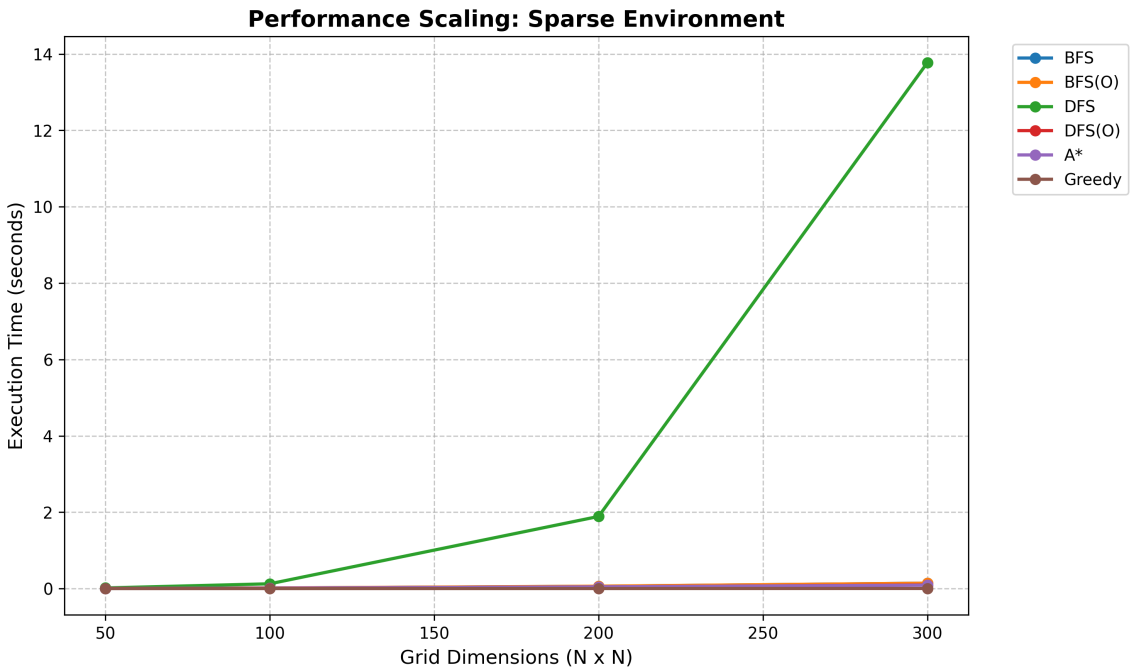

Figure 3: Algorithm performance scaling on a sparse grid (25% walls)

[Distribution: SPARSE]

Size	BFS	BFS(O)	DFS	DFS(O)	A*	Greedy
50x50	0.0038	0.0084	0.0163	0.0033	0.0025	0.0003
100x100	0.0133	0.0156	0.1254	0.0133	0.0107	0.0007
200x200	0.0598	0.0625	1.8890	0.0417	0.0459	0.0015
300x300	0.1413	0.1415	13.7725	0.0959	0.0896	0.0030

Figure 4: Raw execution times: sparse environment

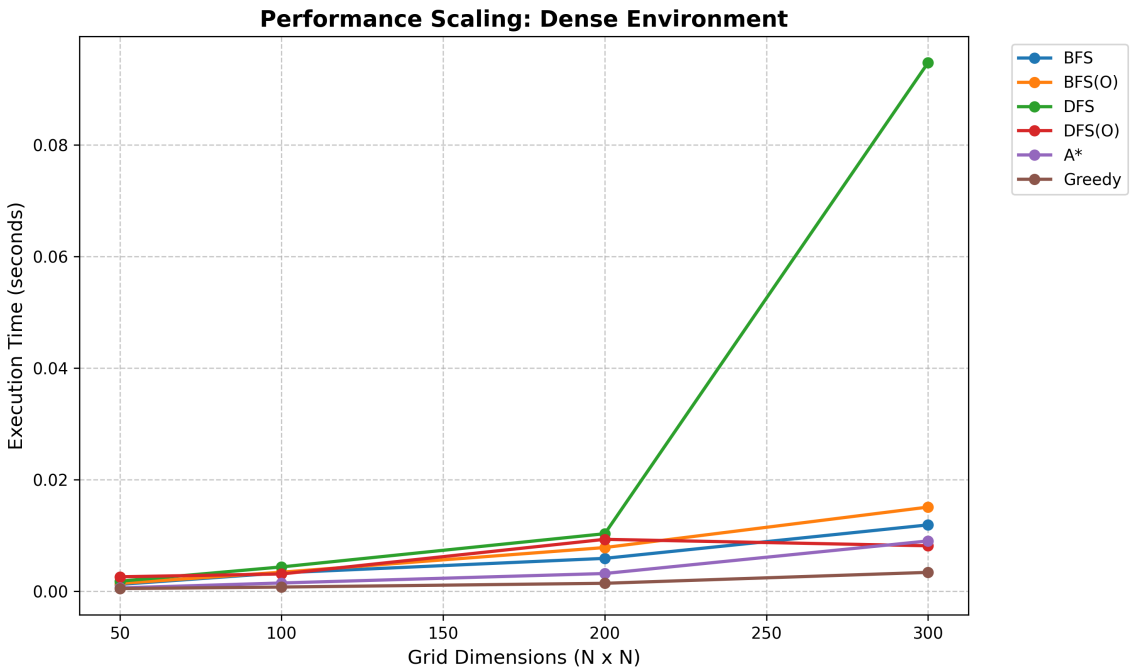

Figure 5: Algorithm performance scaling on a dense grid (45% walls)

[Distribution: DENSE]

Size	BFS	BFS(O)	DFS	DFS(O)	A*	Greedy
50x50	0.0012	0.0014	0.0018	0.0026	0.0006	0.0005
100x100	0.0033	0.0034	0.0044	0.0031	0.0015	0.0008
200x200	0.0059	0.0078	0.0103	0.0093	0.0032	0.0014
300x300	0.0119	0.0151	0.0947	0.0082	0.0090	0.0034

Figure 6: Raw execution times: dense environment

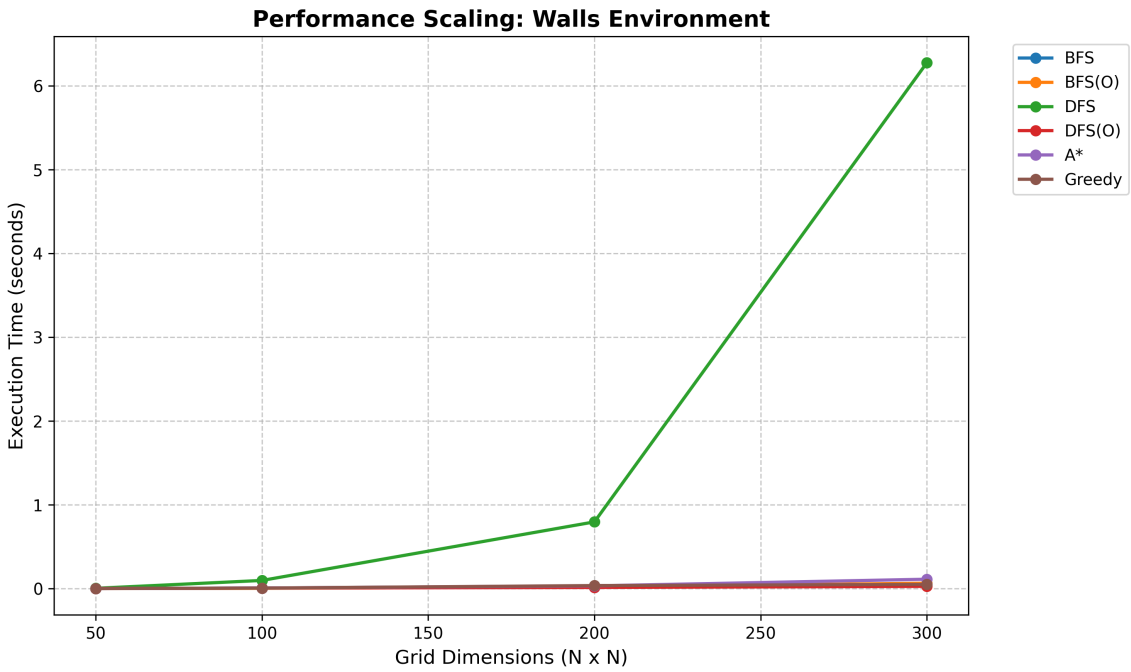

Figure 7: Algorithm performance scaling on strict maze walls

[Distribution: WALLS]

Size	BFS	BFS(O)	DFS	DFS(O)	A*	Greedy
50x50	0.0011	0.0007	0.0036	0.0013	0.0004	0.0010
100x100	0.0069	0.0034	0.0973	0.0064	0.0071	0.0062
200x200	0.0167	0.0219	0.7960	0.0137	0.0326	0.0362
300x300	0.0425	0.0643	6.2747	0.0268	0.1120	0.0499

Figure 8: Raw execution times: maze walls environment

5. CONCLUSION

The empirical analysis clearly demonstrates that the structure of the input data (the grid layout) and the choice of underlying data structures fundamentally dictate search algorithm performance.

The most glaring result is the massive difference between basic BFS and Optimized BFS in open environments. Because basic BFS uses a Python list to pop from index 0, every node extraction forces the array to shift, resulting in exponential time scaling ($O(N)$ operations for thousands of queue items). Replacing it with a `deque` (optimized BFS) drops this to near-zero time overhead. Furthermore, Recursive DFS proved completely unviable for large maps, consistently breaching Python's recursion limit, necessitating the Iterative Stack approach.

Space complexity is a critical factor in graph traversal. In an empty 300×300 grid, BFS expands radically outwards, holding thousands of nodes in its frontier queue simultaneously. By contrast, DFS uses far less memory in open spaces because its stack only grows as deep as its current singular path. However, in a tightly constrained "maze walls" environment, the branching factor shrinks so heavily that BFS's frontier remains incredibly small, minimizing its memory footprint entirely.

The inclusion of A* and Greedy BFS highlights the power of heuristics. In empty and sparse environments, Greedy BFS is practically instant, drawing a straight line to the target. However, it fails to find the optimal path if walls block its way. A* Search represents the perfect balance: it visits significantly fewer nodes than BFS by pulling toward the target (saving space complexity), yet unlike DFS or Greedy, it algorithmically guarantees the absolute shortest path.

In summary, BFS is memory-heavy but reliably optimal; DFS is fast and memory-light in open grids but creates terrible meandering paths; Greedy is lightning fast but easily trapped; and A* is the definitive choice for modern pathfinding, provided the heuristic can be accurately calculated.